

DSS5201 Data Visualisation

Week 7 Tidy Data

David CHEW

Semester 2 AY25/26

Last Meeting: Data Manipulation

Previously, we covered *data manipulation* a.k.a *data wrangling* with `pandas`:

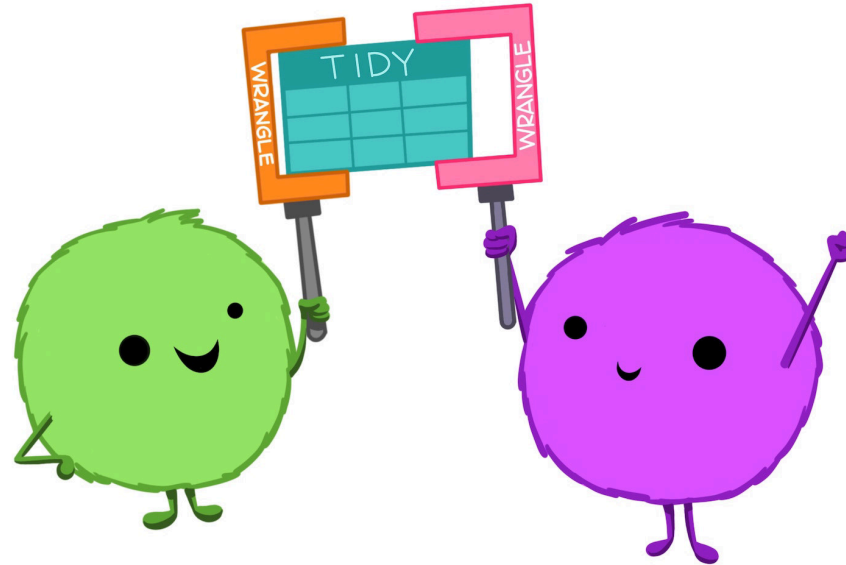
- String and `datetime` manipulation
- Column manipulation
- Row manipulation
- Grouping and group summaries

This Week: Tidy Data

- What is Tidy Data?
- Reshaping: Wide to Long
- Reshaping: Long to Wide
- Splitting a Column

Tidy Data

Tidy Data



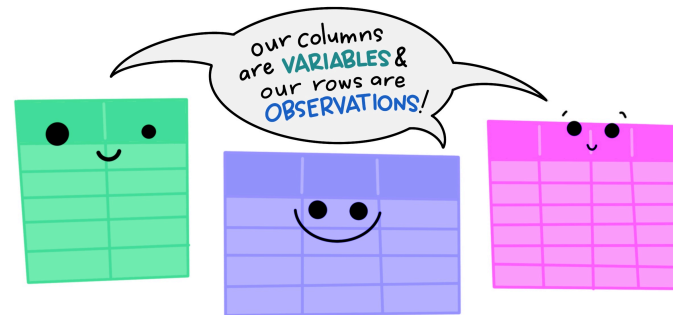
You must have heard of the word “tidy” in your life.

- In data analysis, **tidy data** refers to a standard way of mapping the meaning of data to its structure.
- Getting data into this format requires some upfront work, but it pays off in the long term.

Data Structure

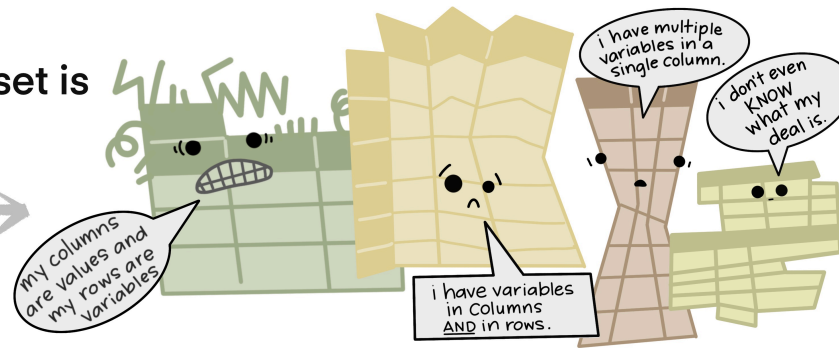
Most statistical data sets are **rectangular tables** made up of rows and columns.

The standard structure of tidy data means that
"tidy datasets are all alike..."



"...but every messy dataset is
messy in its own way."

—HADLEY WICKHAM



Artwork by Allison Horst

Data Semantics

Diagram illustrating the structure of a dataset:

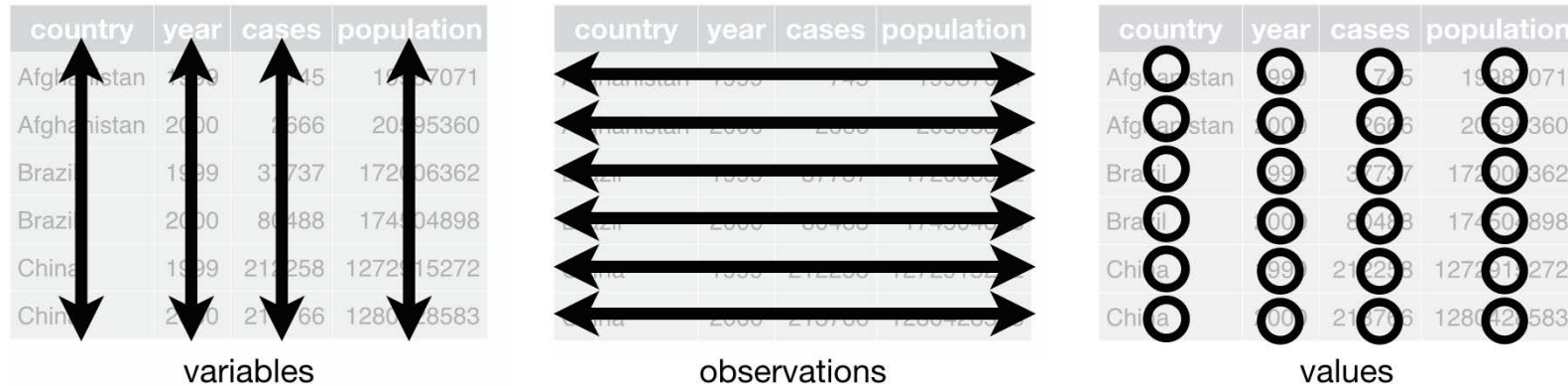
- variables** (columns): ID, NAME, TEAM, STRTWGHT, ENDWGHT
- data values** (rows): 1, 2, 3, 4, 5
- observation** (row 2): 1049, Amelia Serrano, yellow, 145, 124

	ID	NAME	TEAM	STRTWGHT	ENDWGHT
1	1023	David Shaw	red	189	165
2	1049	Amelia Serrano	yellow	145	124
3	1219	Alan Nance	red	210	192
4	1246	Ravi Sinha	yellow	194	177
5	1078	Ashley McKnight	red	127	118

1. A **data set** is a collection of values.
2. Every **value** belongs to a variable (column) and an observation (row).
3. A **variable** contains all values that measure the same underlying attribute across observations.
4. An **observation** contains all values measured across attributes for the same observation unit.

Tidy Data

Definition



Tidy data is a standard way of structuring a data.

There are three interrelated rules:

1. Each variable forms a column.
2. Each observation forms a row.
3. Each cell contains a single value.

Source: Hadley Wickham

Example

Tidy Data?

Take a look at the data below. Is this a tidy data frame?

Stock Prices			
Date	Amazon	Boeing	Google
2009-01-01	174.90	173.55	174.34
2009-01-02	171.42	172.61	170.04

- Data are indeed organised in a rectangular spreadsheet.
- But they **do not** follow the definition of tidy format.

Example

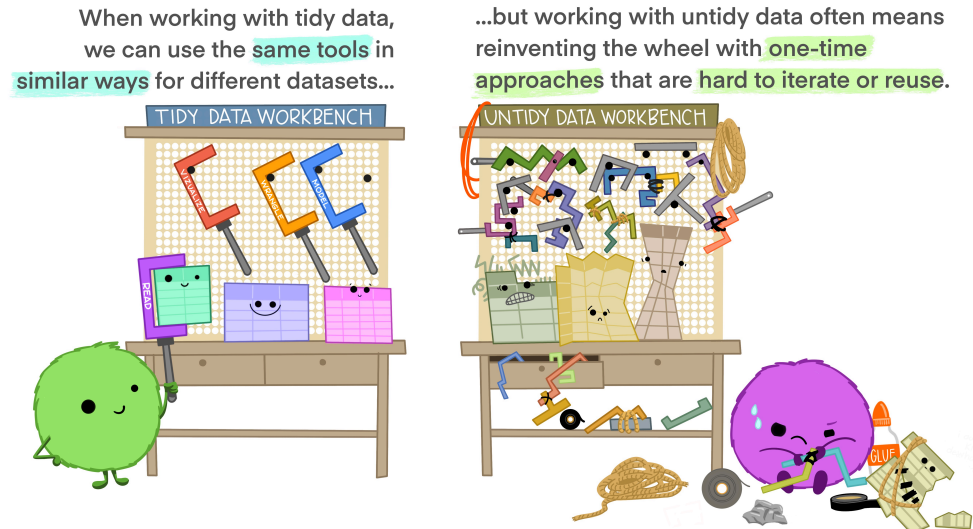
Tidy Data!

1. Each variable should have its own **column**
 - Date, Stock, Price
2. Each observation should form its own **row**
 - Price of a given stock on a given day
3. Each **cell** should contain a single value.

Stock Prices (Tidy)

Date	Stock	Prices
2009-01-01	Amazon	174.90
2009-01-01	Boeing	173.55
2009-01-01	Google	174.30
2009-01-02	Amazon	171.42
2009-01-02	Boeing	172.61
2009-01-02	Google	170.04

Why Tidy Data?



We will *organise data* in a consistent and structured way using the *Principles of Tidy Data*. This makes analysis and visualisation easier.

- Placing variables in column allows the **vectorised** nature of `pandas` functions to work well.
- Most visualisation functions are designed for tidy data.
- A consistent structure means that we don't have to reinvent the wheel.

Tuberculosis Data

We will work with `wk7_tables.xlsx`.

The file contains several spreadsheets on Tuberculosis (TB) cases in three countries during the years 1999 and 2000, each organised in different ways.

Tuberculosis Data

This is Table 1:

```
1 import pandas as pd
2 from itables import show
3 t1 = pd.read_excel('../data/wk7_tables.xlsx', sheet_name = 'table1')
4 show(t1)
```

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

Tuberculosis Data

Table 2 is similar to Table 1, but organised in a different way:

```
1 t2 = pd.read_excel('../data/wk7_tables.xlsx', sheet_name = 'table2')
2 show(t2, lengthMenu=[6, 12], pageLength = 6)
```

6 entries per page

Search:

country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362

Showing 1 to 6 of 12 entries

Tuberculosis Data

Sketch out what you would do for the following:

1. Extract TB cases per country per year.
2. For each country and year, compute the number of cases per 10,000 of population.

Which table is easier to work with, and why?

Tuberculosis Data

Task 1: Number of cases per year

```
1 t1[['country', 'year', 'cases']].head(6)
```

	country	year	cases
0	Afghanistan	1999	745
1	Afghanistan	2000	2666
2	Brazil	1999	37737
3	Brazil	2000	80488
4	China	1999	212258
5	China	2000	213766

Tuberculosis Data

Task 1: Number of cases per year

```
1 t2[t2['type'] == 'cases'].head(6)
```



	country	year	type	count
0	Afghanistan	1999	cases	745
2	Afghanistan	2000	cases	2666
4	Brazil	1999	cases	37737
6	Brazil	2000	cases	80488
8	China	1999	cases	212258
10	China	2000	cases	213766

Tuberculosis Data

Task 2: Number of cases per 10,000 per year

```
1 t1['cases_pc'] = t1['cases']/(t1['population']/10000)
2 t1.head(6)
```

	country	year	cases	population	cases_pc
0	Afghanistan	1999	745	19987071	0.372741
1	Afghanistan	2000	2666	20595360	1.294466
2	Brazil	1999	37737	172006362	2.193930
3	Brazil	2000	80488	174504898	4.612363
4	China	1999	212258	1272915272	1.667495
5	China	2000	213766	1280428583	1.669488

Straightforward with `t1`. How to do this with `t2`?

Tuberculosis Data

Task 2: Number of cases per 10,000 per year

- With `t2`, TB cases and population are stored under the same column `count`.
- `pandas` functions apply operations column-wise, cannot be used directly.
- We will create a `population` column using `.transform()`.

```
1 t2['population'] = t2.groupby(['country', 'year'])[['count']].transform('max')
2 t2.head()
```

	country	year	type	count	population
0	Afghanistan	1999	cases	745	19987071
1	Afghanistan	1999	population	19987071	19987071
2	Afghanistan	2000	cases	2666	20595360
3	Afghanistan	2000	population	20595360	20595360
4	Brazil	1999	cases	37737	172006362

Tuberculosis Data

Task 2: Number of cases per 10,000 per year

With this format, we can compute the infection rates.

```
1 t2['cases_pc'] = t2['count']/(t2['population']/10000)
2 t2.head()
```

	country	year	type	count	population	cases_pc
0	Afghanistan	1999	cases	745	19987071	0.372741
1	Afghanistan	1999	population	19987071	19987071	10000.000000
2	Afghanistan	2000	cases	2666	20595360	1.294466
3	Afghanistan	2000	population	20595360	20595360	10000.000000
4	Brazil	1999	cases	37737	172006362	2.193930

Tuberculosis Data

Task 2: Number of cases per 10,000 per year

After that, we extract the rows we need.

```
1 t2[t2['type'] == 'cases'].head(6)
```

	country	year	type	count	population	cases_pc
0	Afghanistan	1999	cases	745	19987071	0.372741
2	Afghanistan	2000	cases	2666	20595360	1.294466
4	Brazil	1999	cases	37737	172006362	2.193930
6	Brazil	2000	cases	80488	174504898	4.612363
8	China	1999	cases	212258	1272915272	1.667495
10	China	2000	cases	213766	1280428583	1.669488

How to Tidy Up Data

Data can be untidy in many different ways. The two most common issues are

- Column headers are values, instead of variable names.
 - We can solve this with `melt()`.
- Multiple variables are stored in one column.
 - We can solve this with `pivot()` or `pivot_table()`.

Most real analyses will require at least a little tidying.

Reshaping: Wide to Long

Tuberculosis Data

Table 4a

We will continue working with the Tuberculosis Data, `wk7_tables.xlsx`.

```
1 import pandas as pd
2 table4a = pd.read_excel('../data/wk7_tables.xlsx', sheet_name = 'table4a')
3 table4a.head()
```

	country	1999	2000
0	Afghanistan	745	2666
1	Brazil	37737	80488
2	China	212258	213766

There are three variables in Table 4a:

- Country, with 3 values.
- Year, with 2 values.
- TB cases, with 6 values – one per country and year.

Tuberculosis Data

Table 4a

	country	1999	2000
0	Afghanistan	745	2666
1	Brazil	37737	80488
2	China	212258	213766

This table is not tidy.

1. Year is stored as column names (1999 and 2000), instead of in a column.

Okay for display, but it is not tidy.

2. TB cases spread across columns and rows, instead of values in a column.

We can resolve these issues and tidy up the table with `.melt()`.

Reshaping Data from Wide to Long with `.melt()`

We can reshape data from *wide* to *long* using the function `.melt()`. The key arguments are:

1. `id_vars`: Column(s) to keep fixed (identifiers).
 - In Table 4a, it is the `country` column.
2. `value_vars`: Columns to melt into long format.
 - In Table 4a, it should be the `1999` and `2000` columns.
 - If not provided, it will use all non `id_vars` columns.
3. `var_name`: Name for the new column that holds the melted column names.
 - We can name this column `year`.
4. `value_name`: Name for the new column that holds the values.
 - We can name this column `cases`.

Tuberculosis Data

Table 4a: How to .melt()

```
1 table4a_tidy = table4a.melt(  
2     id_vars = 'country', value_vars = ['1999', '2000'],  
3     var_name = 'year', value_name = 'cases')
```

country	year	cases
Afghanistan	1999	745
Afghanistan	2000	2666
Brazil	1999	37737
Brazil	2000	80488
China	1999	212258
China	2000	213766

table4a_tidy

country	1999	2000
Afghanistan	745	2666
Brazil	37737	80488
China	212258	213766

table4a

Tuberculosis Data

Table 4a: How to .melt()

```
1 table4a_tidy = table4a.melt(  
2     id_vars = 'country', value_vars = ['1999', '2000'],  
3     var_name = 'year', value_name = 'cases')  
4 show(table4a_tidy)
```

country	year	cases
Afghanistan	1999	745
Brazil	1999	37737
China	1999	212258
Afghanistan	2000	2666
Brazil	2000	80488
China	2000	213766

The resulting table is tidy!

Reshaping: Long to Wide

Tuberculosis Data

Table 2

Let's look at Table 2 again.

```
1 table2 = pd.read_excel('../data/wk7_tables.xlsx', sheet_name = 'table2')
2 table2.head(6)
```

	country	year	type	count
0	Afghanistan	1999	cases	745
1	Afghanistan	1999	population	19987071
2	Afghanistan	2000	cases	2666
3	Afghanistan	2000	population	20595360
4	Brazil	1999	cases	37737
5	Brazil	1999	population	172006362

- The ideal observation unit should be **country-year**.
- The **type** column contains two types of measurements for each ideal observation unit.

Reshaping Data from Long to Wide with `.pivot()`

We can reshape data from *long* to *wide* using the function `.pivot()`. The key arguments are:

1. `index`: Column(s) as identifiers – will be used as the new row index.
 - In Table 2, these will be `country` and `year`.
2. `columns`: Column that currently contains variable names – unique values will be used to make new variable names.
 - In Table 2, this will be `type`.
3. `values`: Column that currently contains multiple values – will be used to fill the cells in the new table.
 - In Table 2, this will be `count`.

Tuberculosis Data

Table 2: How to .pivot()

```
1 table2_tidy = table2.pivot(  
2     index = ['country', 'year'],  
3     columns = 'type', values = 'count')
```

country	year	type	count
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898
China	1999	cases	212258
China	1999	population	1272915272
China	2000	cases	213766
China	2000	population	1280428583

table2

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

table2_tidy

Tuberculosis Data

Table 2: How to .pivot()

```
1 table2_tidy = table2.pivot(  
2     index = ['country', 'year'],  
3     columns = 'type', values = 'count')  
4 table2_tidy
```

	type	cases	population
country	year		
Afghanistan	1999	745	19987071
	2000	2666	20595360
Brazil	1999	37737	172006362
	2000	80488	174504898
China	1999	212258	1272915272
	2000	213766	1280428583

Note that `country` and `year` are used as row index in the new table.

Tuberculosis Data

Table 2: Results after `.pivot()`

	type	cases	population
country	year		
Afghanistan	1999	745	19987071
	2000	2666	20595360
Brazil	1999	37737	172006362
	2000	80488	174504898
China	1999	212258	1272915272
	2000	213766	1280428583

Rows: Hierarchical index (multi-index).

- Level 1: `country`.
- Level 2: `year`.
- Rows are grouped by country, with years nested within.

Columns:

- Single index with index name `type`.

Next Steps: Flatten the row index and remove the name of the column index for downstream analysis.

Tuberculosis Data

Table 2: Reset Row Indices

1. Reset row indices to turn `country` and `year` back into regular columns.

```
1 table2_tidy = table2_tidy.reset_index()  
2 table2_tidy
```

type	country	year	cases	population
0	Afghanistan	1999	745	19987071
1	Afghanistan	2000	2666	20595360
2	Brazil	1999	37737	172006362
3	Brazil	2000	80488	174504898
4	China	1999	212258	1272915272
5	China	2000	213766	1280428583

Tuberculosis Data

Table 2: Remove Column Index Name

2. Remove the name of the column index (`type`).

```
1 table2_tidy.columns.name = None
2 table2_tidy
```

	country	year	cases	population
0	Afghanistan	1999	745	19987071
1	Afghanistan	2000	2666	20595360
2	Brazil	1999	37737	172006362
3	Brazil	2000	80488	174504898
4	China	1999	212258	1272915272
5	China	2000	213766	1280428583

Tuberculosis Data

Table 2: Complete .pivot() Operation

	country	year	type	count
0	Afghanistan	1999	cases	745
1	Afghanistan	1999	population	19987071
2	Afghanistan	2000	cases	2666
3	Afghanistan	2000	population	20595360

The complete operation of .pivot()

`.pivot()`

	type	cases	population
country	year		
Afghanistan	1999	745	19987071
	2000	2666	20595360
Brazil	1999	37737	172006362
	2000	80488	174504898
China	1999	212258	1272915272
	2000	213766	1280428583

`.reset_index()`

`.column.name = None`

	type	country	year	cases	population
0		Afghanistan	1999	745	19987071
1		Afghanistan	2000	2666	20595360
2		Brazil	1999	37737	172006362
3		Brazil	2000	80488	174504898
4		China	1999	212258	1272915272



	country	year	cases	population
0	Afghanistan	1999	745	19987071
1	Afghanistan	2000	2666	20595360
2	Brazil	1999	37737	172006362
3	Brazil	2000	80488	174504898
4	China	1999	212258	1272915272

Tuberculosis Data

Table 2: Complete .pivot() Operation

The complete operation is:

```
1 table2_tidy = table2.pivot(  
2     index = ['country', 'year'],  
3     columns = 'type', values = 'count')  
4 table2_tidy = table2_tidy.reset_index()  
5 table2_tidy.columns.name = None  
6 table2_tidy
```

	country	year	cases	population
0	Afghanistan	1999	745	19987071
1	Afghanistan	2000	2666	20595360
2	Brazil	1999	37737	172006362
3	Brazil	2000	80488	174504898
4	China	1999	212258	1272915272
5	China	2000	213766	1280428583

Tuberculosis Data

Table 2: Alternative Method

`.pivot_table()` is a generalisation of `.pivot()`.

- It has a similar syntax, but more flexible.

```
1 table2_tidy1 = table2.pivot_table(  
2     index = ['country', 'year'],  
3     columns = 'type', values = 'count')  
4 table2_tidy1 = table2_tidy1.reset_index()  
5 table2_tidy1.columns.name = None  
6 table2_tidy1
```

	country	year	cases	population
0	Afghanistan	1999	745.0	1.998707e+07
1	Afghanistan	2000	2666.0	2.059536e+07
2	Brazil	1999	37737.0	1.720064e+08
3	Brazil	2000	80488.0	1.745049e+08
4	China	1999	212258.0	1.272915e+09
5	China	2000	213766.0	1.280429e+09

`.melt()` versus `.pivot()`

- `.melt()` makes a data frame tall and narrow.
- `.pivot()` and `.pivot_table()` make a data frame wide and short.

Long versus Wide

Just because a dataset is long or wide **does not** mean it is tidy.

A long table that is **not** tidy.

country	year	variable	value
Afghanistan	1999	cases	745
Afghanistan	1999	population	19987071
Afghanistan	2000	cases	2666
Afghanistan	2000	population	20595360
Brazil	1999	cases	37737
Brazil	1999	population	172006362
Brazil	2000	cases	80488
Brazil	2000	population	174504898

A wide table that is **not** tidy.

country	cases_1999	cases_2000	population_1999	population_2000
Afghanistan	745	2666	19987071	20595360
Brazil	37737	80488	172006362	174504898

It depends on how information is organised – whether the structure aligns with tidy data principles.

Splitting a Column

Tuberculosis Data

Table 3

The same TB data could also be stored in a structure like this — counts and population are recorded as a *rate*, separated by a forward slash /.

```
1 table3 = pd.read_excel('../data/wk7_tables.xlsx', sheet_name = 'table3')
2 table3
```

	country	year	rate
0	Afghanistan	1999	745/19987071
1	Afghanistan	2000	2666/20595360
2	Brazil	1999	37737/172006362
3	Brazil	2000	80488/174504898
4	China	1999	212258/1272915272
5	China	2000	213766/1280428583

Tuberculosis Data

Table 3: `.split()` a Column

To split the `rate` column, we use `.split()` with the `.str` accessor:

- `expand = True` splits the `rate` column into two.
- We will also convert the new columns to integer type.

```
1 table3_tidy = table3.copy()
2 table3_tidy[['cases', 'population']] = (
3     table3_tidy['rate']
4     .str.split('/', expand = True)
5     .astype(int)
6 )
7 table3_tidy.head(4)
```

	country	year	rate	cases	population
0	Afghanistan	1999	745/19987071	745	19987071
1	Afghanistan	2000	2666/20595360	2666	20595360
2	Brazil	1999	37737/172006362	37737	172006362
3	Brazil	2000	80488/174504898	80488	174504898

Tuberculosis Data

Table 3: `.split()` a Column

```
1 table3_tidy[['cases', 'population']] = (  
2     table3_tidy['rate']  
3     .str.split('/', expand = True)  
4     .astype(int)  
5 )  
6 table3_tidy = table3_tidy.drop(columns = 'rate')
```

The diagram illustrates the transformation of the 'rate' column in 'table3' into 'cases' and 'population' columns in 'table3_tidy'. Arrows show the mapping from the 'rate' column (containing values like '745 / 19987071') to the 'cases' and 'population' columns in the resulting 'table3_tidy'.

country	year	rate
Afghanistan	1999	745 / 19987071
Afghanistan	2000	2666 / 20595360
Brazil	1999	37737 / 172006362
Brazil	2000	80488 / 174504898
China	1999	212258 / 1272915272
China	2000	213766 / 1280428583

table3

country	year	cases	population
Afghanistan	1999	745	19987071
Afghanistan	2000	2666	20595360
Brazil	1999	37737	172006362
Brazil	2000	80488	174504898
China	1999	212258	1272915272
China	2000	213766	1280428583

table3_tidy

Tuberculosis Data

Table 3: Complete .split() Operation

The complete operation is:

```
1 table3_tidy = table3.copy()
2 table3_tidy[['cases', 'population']] = (
3     table3_tidy['rate']
4     .str.split('/', expand = True)
5     .astype(int)
6 )
7 table3_tidy = table3_tidy.drop(columns = 'rate')
8 table3_tidy.head(4)
```

	country	year	cases	population
0	Afghanistan	1999	745	19987071
1	Afghanistan	2000	2666	20595360
2	Brazil	1999	37737	172006362
3	Brazil	2000	80488	174504898

Summary

Summary

The principles of tidy data may seem very obvious now. You wonder if you will ever encounter a data set that is not tidy.

Unfortunately, you will – most data you encounter will be untidy.

- Most people are not familiar with the principles of tidy data, and it is hard to derive them yourself unless you spent a lot of time working with data.
- Data are often organised to facilitate some use, other than analysis. For example, they can be organised to make data entry as easy as possible.
- This means that for most real analyses, you will need to do some data tidying.

Summary

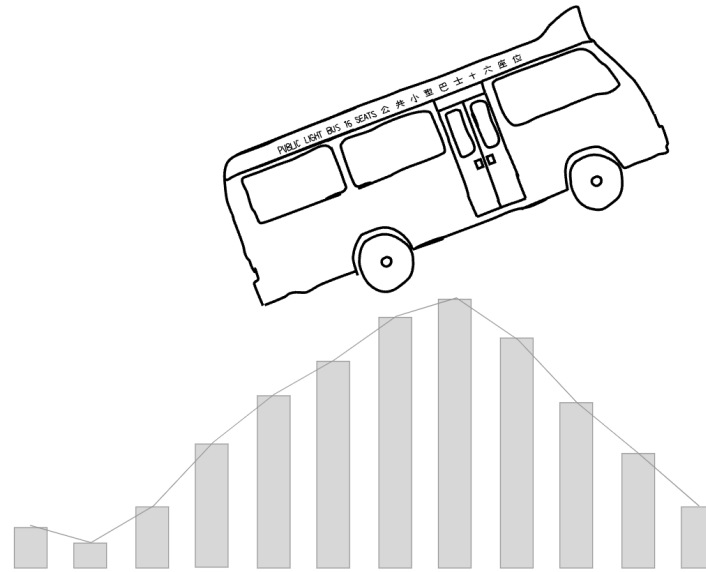
The first step of tidying data is always to figure out what are the variables and observations.

- Sometimes this is easy.
- Other times you will need to consult the people who provided the data.

The second step is to resolve (one of) two common problems:

-  One variable spreads across multiple columns.
-  One observation scatters across multiple rows.

Alternative Libraries



Some other libraries that make data manipulation simple and fast:

- `siuba` and `dfply` offer user-friendly experience that mimics R's `dplyr` syntax.
- If you come to Python from R, you will find these libraries very easy to use.

Note: For information only. `siuba` and `dfply` will not be tested in exams.